

Order Management System

Spring Boot · JPA · Hibernate · REST API

Ramchandra Shipalkar

ADT24SOCBD212

Academic Year 2025 – 26

MIT ART, DESIGN AND TECHNOLOGY UNIVERSITY

School of Computing · LY-IT-CORE

1. Overview

A RESTful web application built with Spring Boot that manages the full lifecycle of customers, products, and orders. The system implements layered architecture, JPA entity relationships, and custom query methods, serving as a practical demonstration of enterprise backend development patterns.

2. Technologies

Technology	Purpose
Java 21	Core programming language
Spring Boot 3	Application framework & embedded Tomcat
Spring Data JPA	Repository abstraction and query derivation
Hibernate	ORM engine — entity ↔ table mapping
H2 Database	In-memory relational database
Lombok	Boilerplate elimination (@Data, @Getter, @Setter)
Maven	Build tool and dependency management
Postman	REST API testing and verification

3. Architecture

Standard three-layer Spring MVC structure:

Layer	Classes	Responsibility
Controller	CustomerController, OrderController, ProductController	Route HTTP requests
Service	CustomerService, OrderService, ProductService	Business logic
Repository	CustomerRepository, OrderRepository, ProductRepository	Data access via JPA
Entity	Customer, Product, Orders	Database-mapped objects
DTO	CustomerDto	Data transfer / projection

4. Entity Design

Customer (table: SwiggyCustomers)

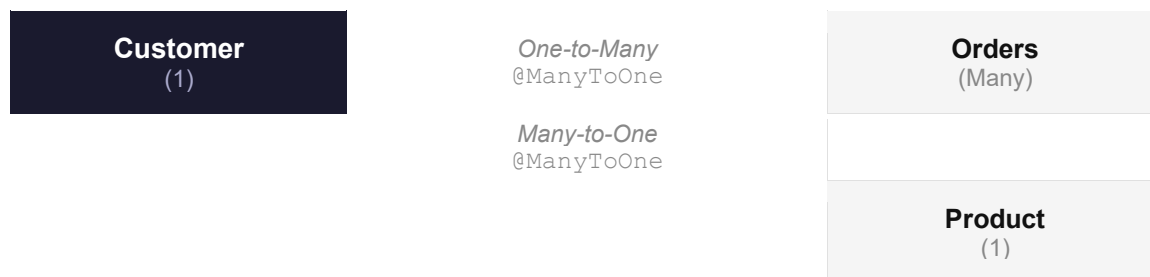
Field	Type	Constraint
customerId	Long	PK · Auto-generated
name	String	NOT NULL
email	String	UNIQUE
city	String	—
age	int	—
bankUserName	String	—
password	int	—

Product (table: product)

Field	Type	Constraint
productId	Long	PK · Auto-generated
productName	String	—
price	double	—
category	String	—
quantity	int	—

Orders (table: Orders)

Field	Type	Notes
orderId	Long	PK · Auto-generated
quantityOrdered	int	—
totalPrice	double	price × qty (computed in service)
customer	Customer	FK → SwiggyCustomers (@ManyToOne)
product	Product	FK → product (@ManyToOne)

Relationship

5. API Endpoints

Customer /customers

Method	Endpoint	Description
GET	/customers	Get all customers
GET	/customers/{id}	Get customer by ID
GET	/customers/email/{email}	Get by email
GET	/customers/city/{city}	Get by city
GET	/customers/age/{age}	Get where age > value
POST	/customers	Create customer
POST	/customers/list	Bulk create customers
PUT	/customers/{id}	Update customer
DELETE	/customers/{id}	Delete customer

Product /products

Method	Endpoint	Description
GET	/products	Get all products
GET	/products/{id}	Get product by ID
GET	/products/category/{cat}	Get by category
POST	/products	Add product
POST	/products/list	Bulk add products
PUT	/products/{id}	Update product
DELETE	/products/{id}	Delete product

Orders /orders

Method	Endpoint	Description
GET	/orders	Get all orders
POST	/orders/{custId}/{prodId}/{qty}	Place order · total = price × qty

6. API Testing — Postman Screenshots

All endpoints tested via Postman at <http://localhost:8081>. Responses verified 200 OK with correct JSON structure. Screenshots taken from Ram Shipalkar's Workspace.

Customer API /customers

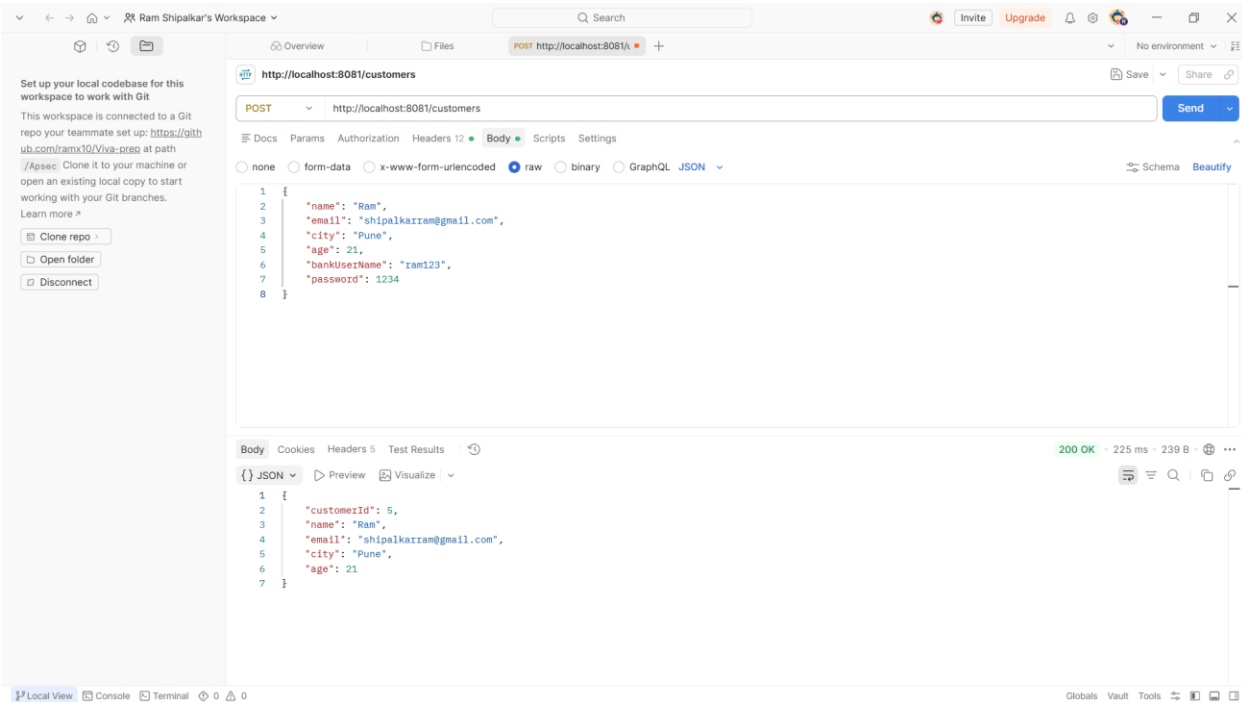


Figure 1 · POST /customers — Create single customer (Ram Shipalkar, Pune) · 200 OK

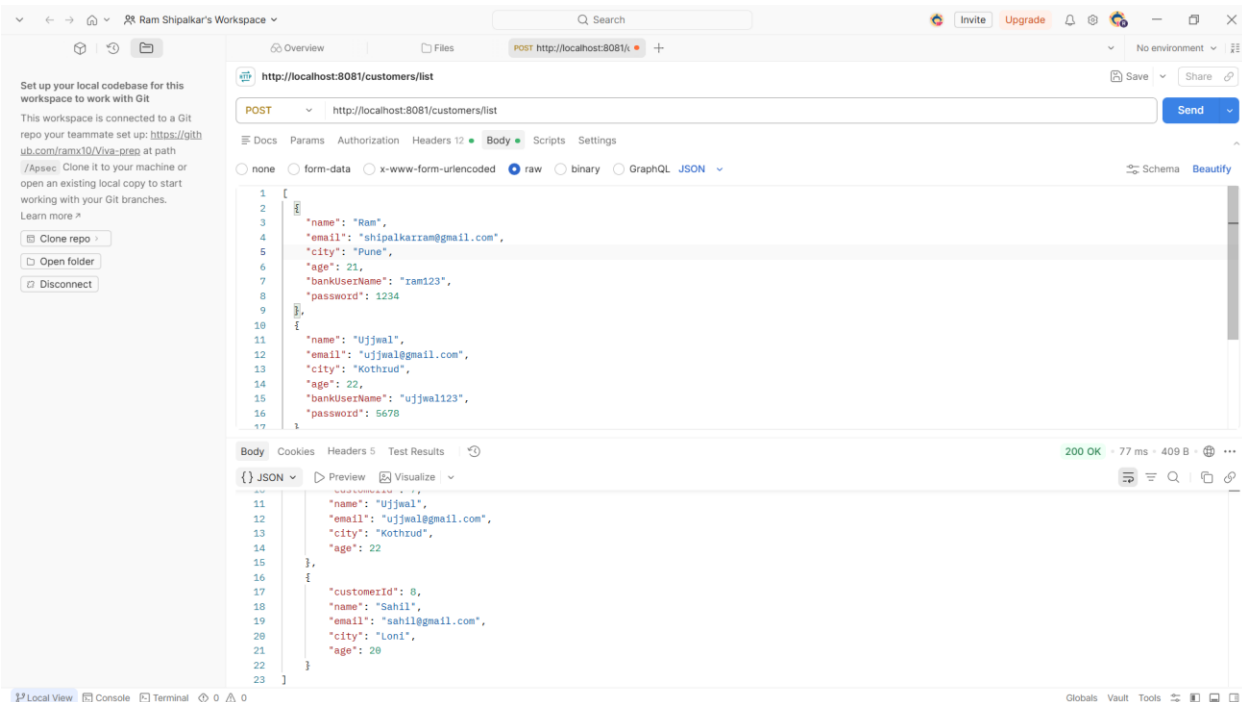


Figure 2 · POST /customers/list — Bulk create customers (Ram, Ujjwal, Sahil) · 200 OK

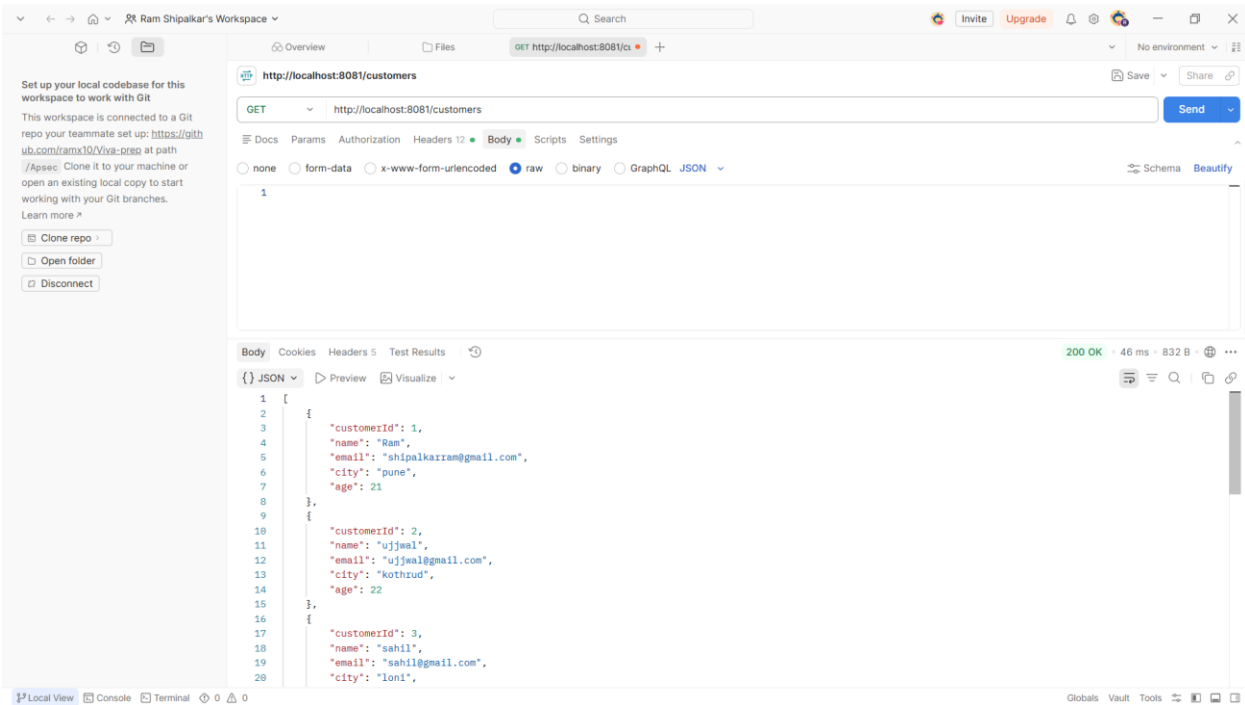


Figure 3 · GET /customers — Retrieve all customers · 200 OK

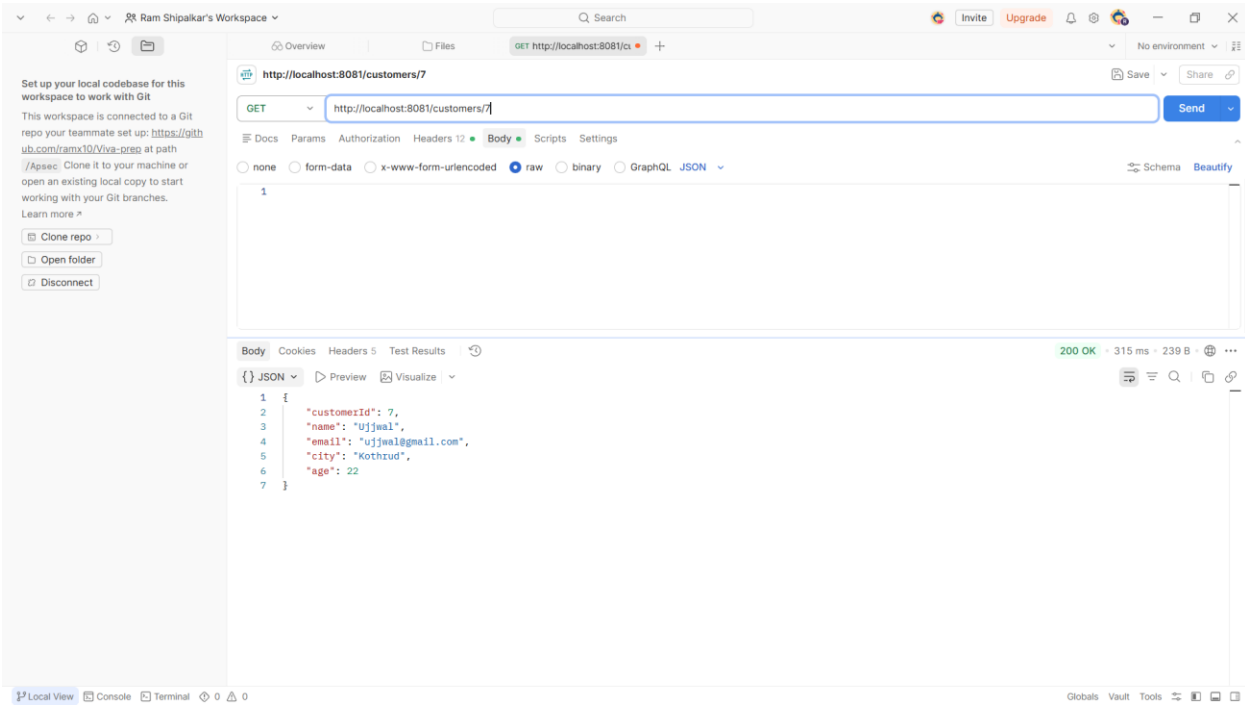


Figure 4 · GET /customers/7 — Retrieve customer by ID (Ujjwal, Kothrud) · 200 OK

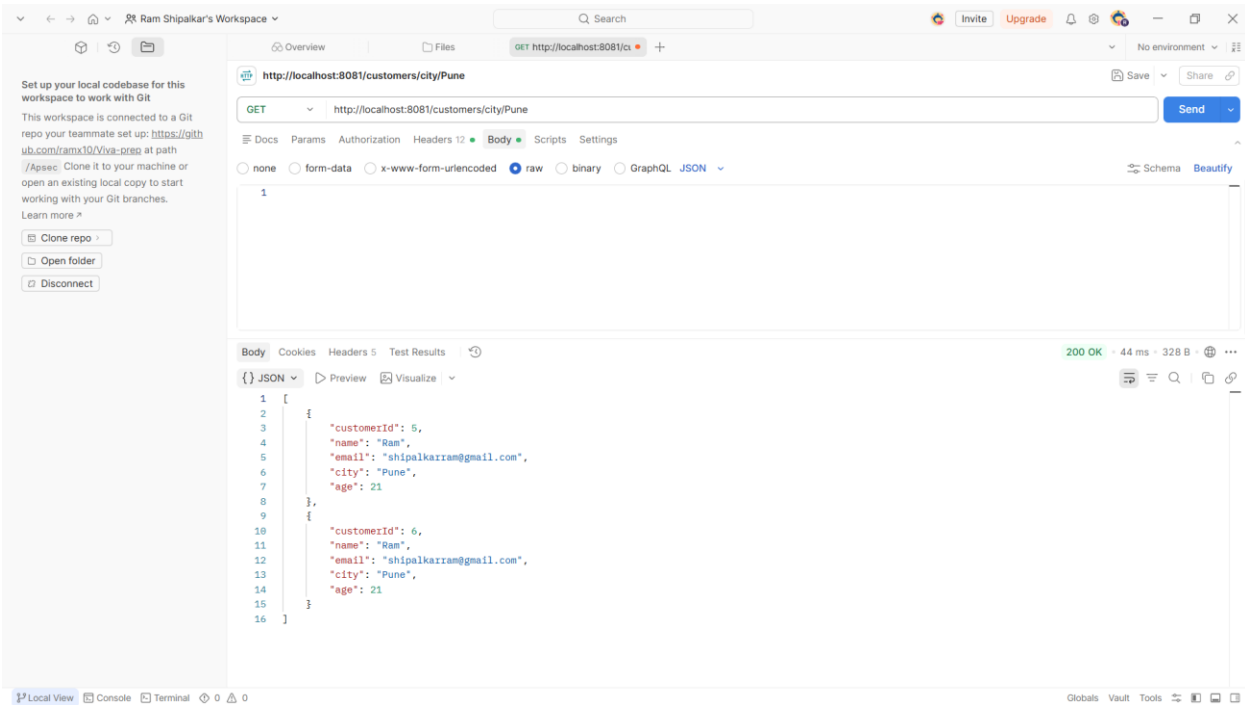


Figure 5 · GET /customers/city/Pune — Retrieve customers by city · 200 OK

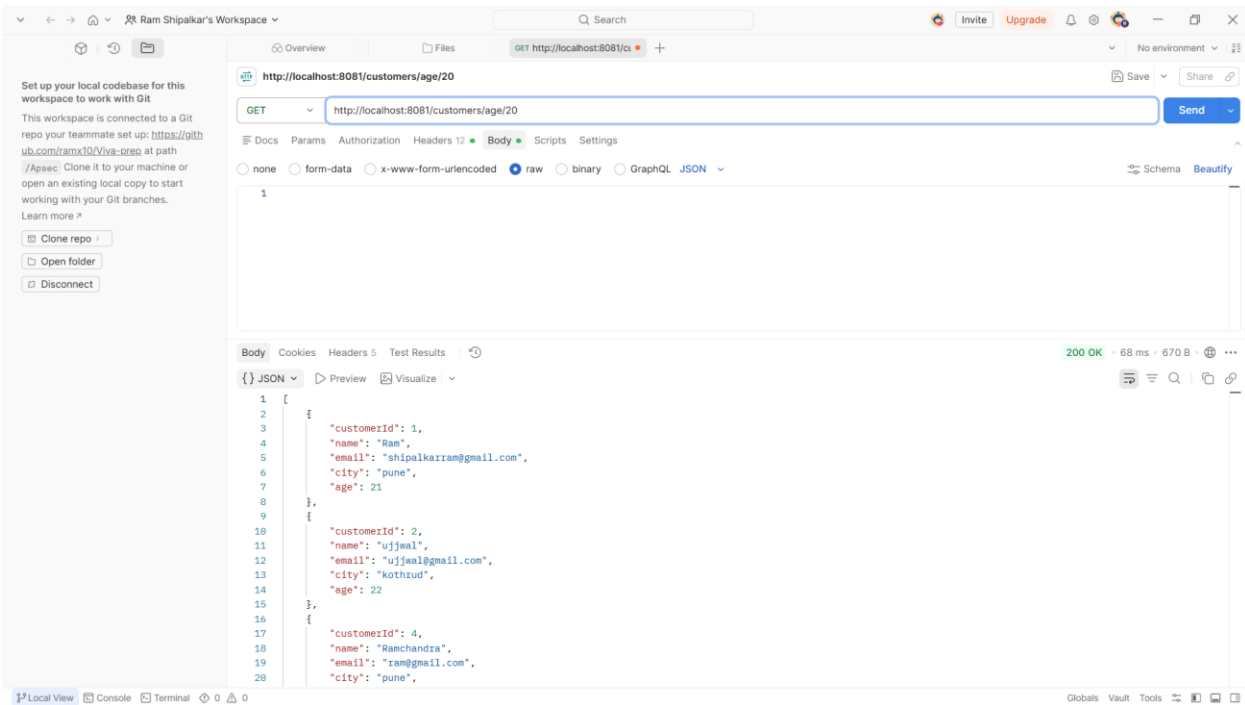


Figure 6 · GET /customers/age/20 — Retrieve customers with age > 20 · 200 OK

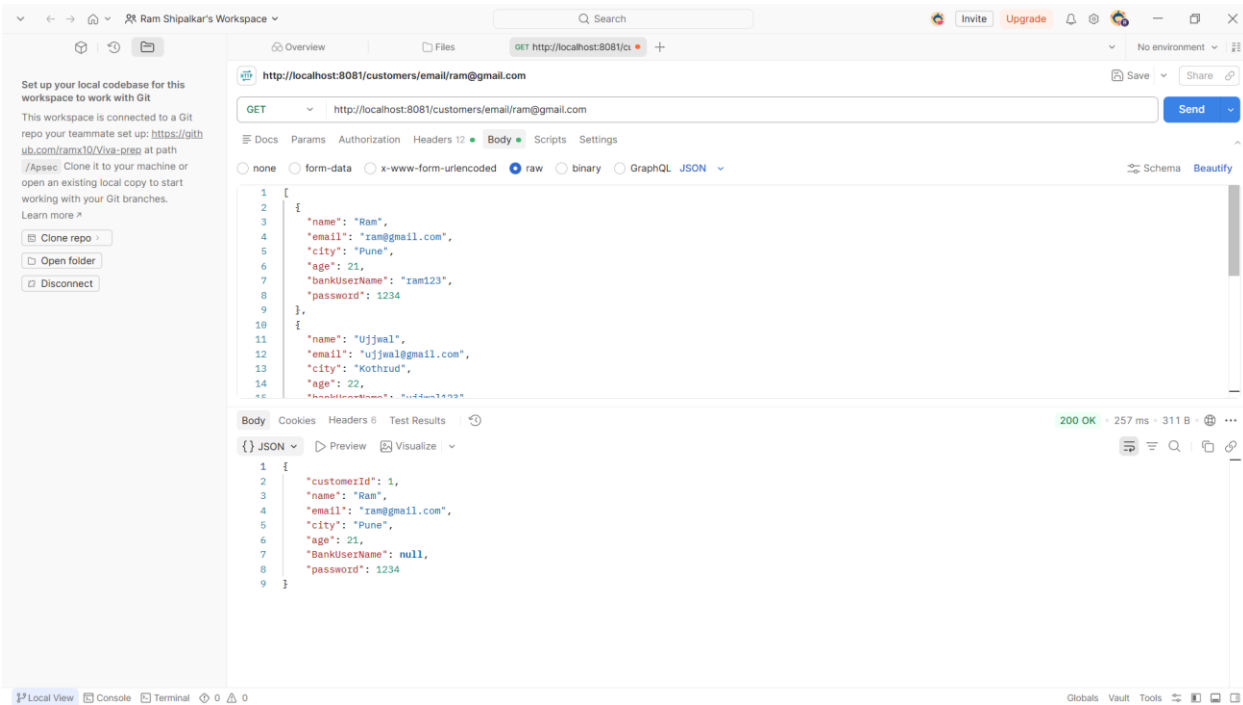


Figure 7 · GET /customers/email/ram@gmail.com — Retrieve customer by email · 200 OK

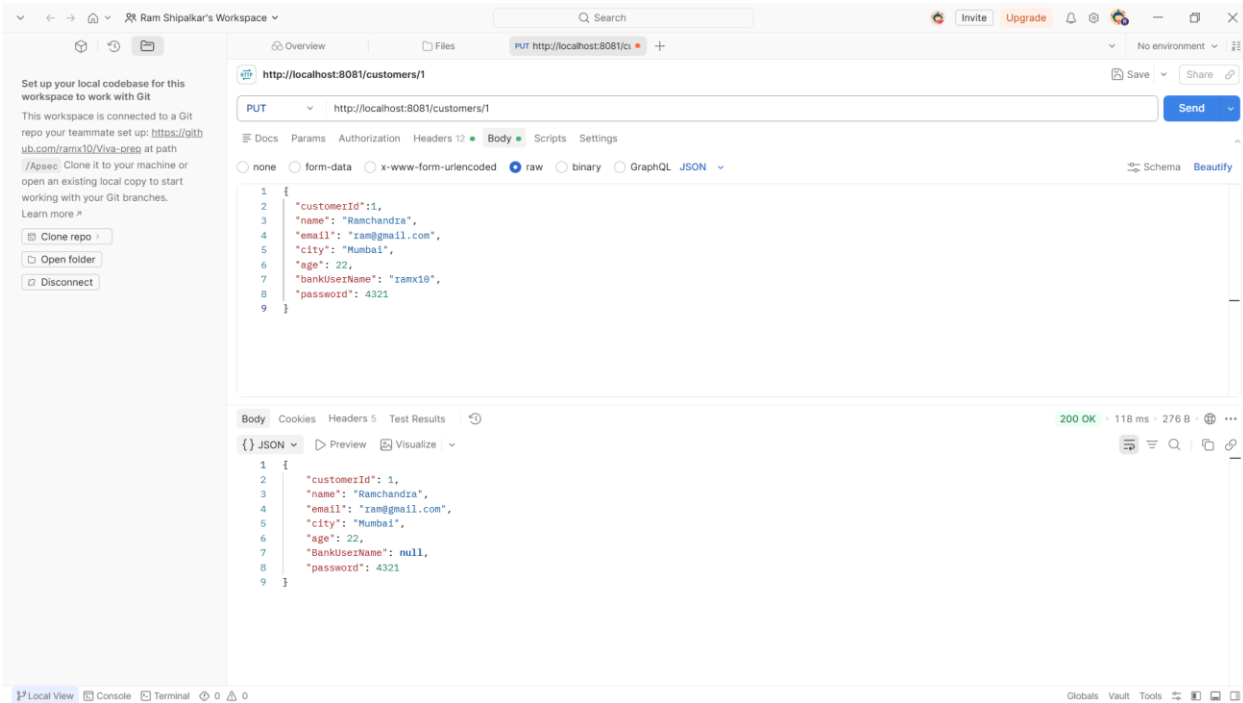


Figure 8 · PUT /customers/1 — Update customer (Ramchandra, Mumbai) · 200 OK

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8081/customers/2`
- Method:** `DELETE`
- Body:** A JSON object representing a customer:

```
1 {
2   "customerId": 1,
3   "name": "Ramchandra",
4   "email": "ram@gmail.com",
5   "city": "Mumbai",
6   "age": 22,
7   "bankUserName": "ramx10",
8   "password": "4321"
9 }
```
- Response:** `200 OK` with a status of `55 ms` and `121 B`.

The interface also includes a sidebar with options like "Clone repo", "Open folder", and "Disconnect". The bottom status bar shows "Local View", "Console", "Terminal", and "0 0".

Figure 9 · DELETE /customers/2 — Delete customer by ID · 200 OK

Product API /products

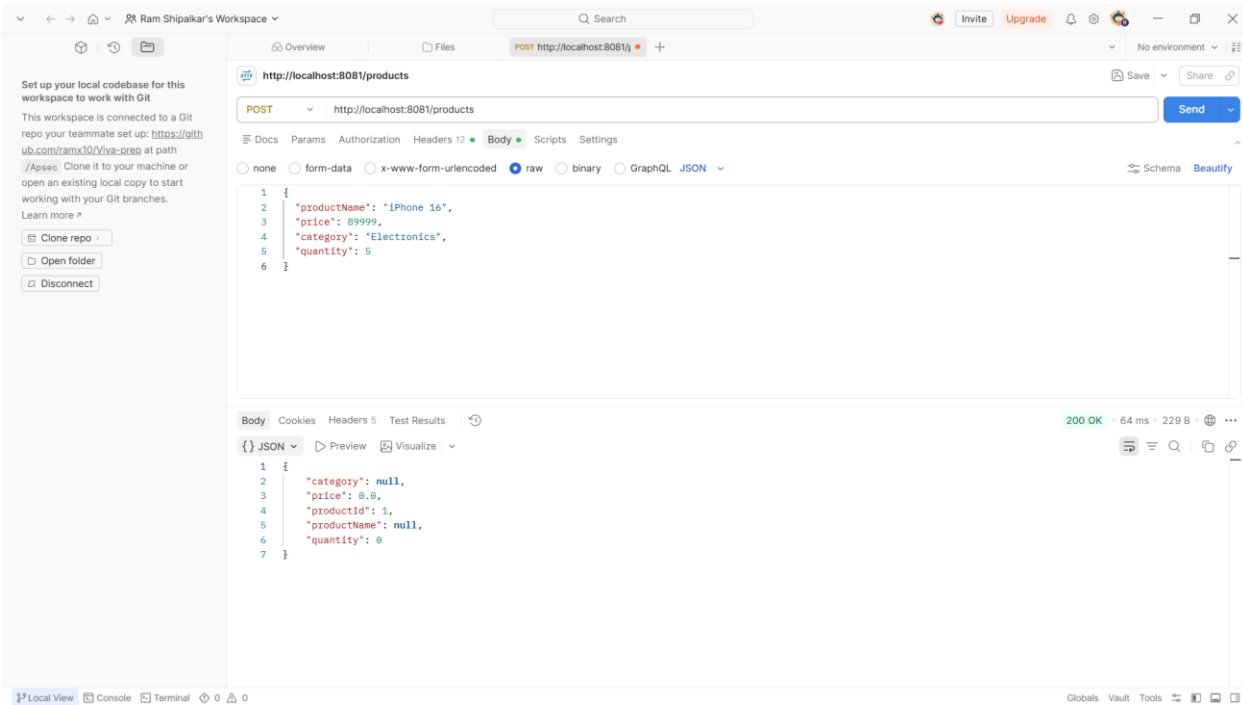


Figure 10 · POST /products — Add single product (iPhone 16, Electronics) · 200 OK

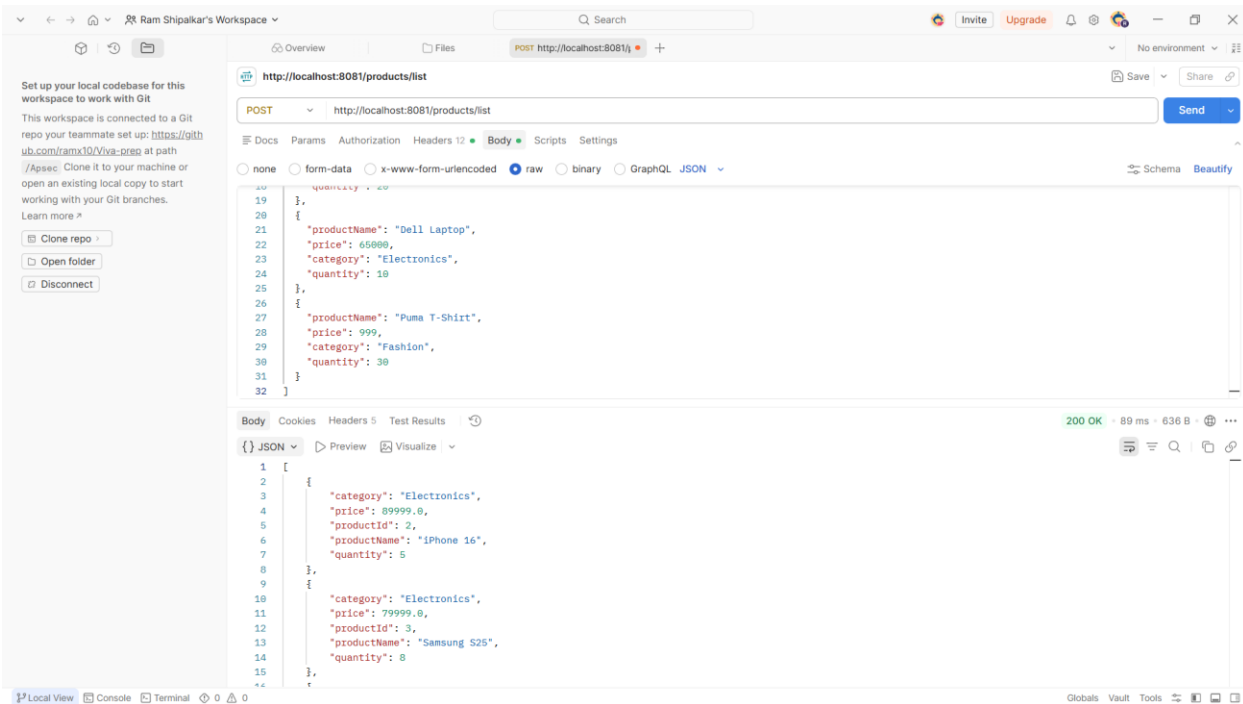


Figure 11 · POST /products/list — Bulk add products (iPhone 16, Samsung S25, Dell Laptop, Puma T-Shirt) · 200 OK

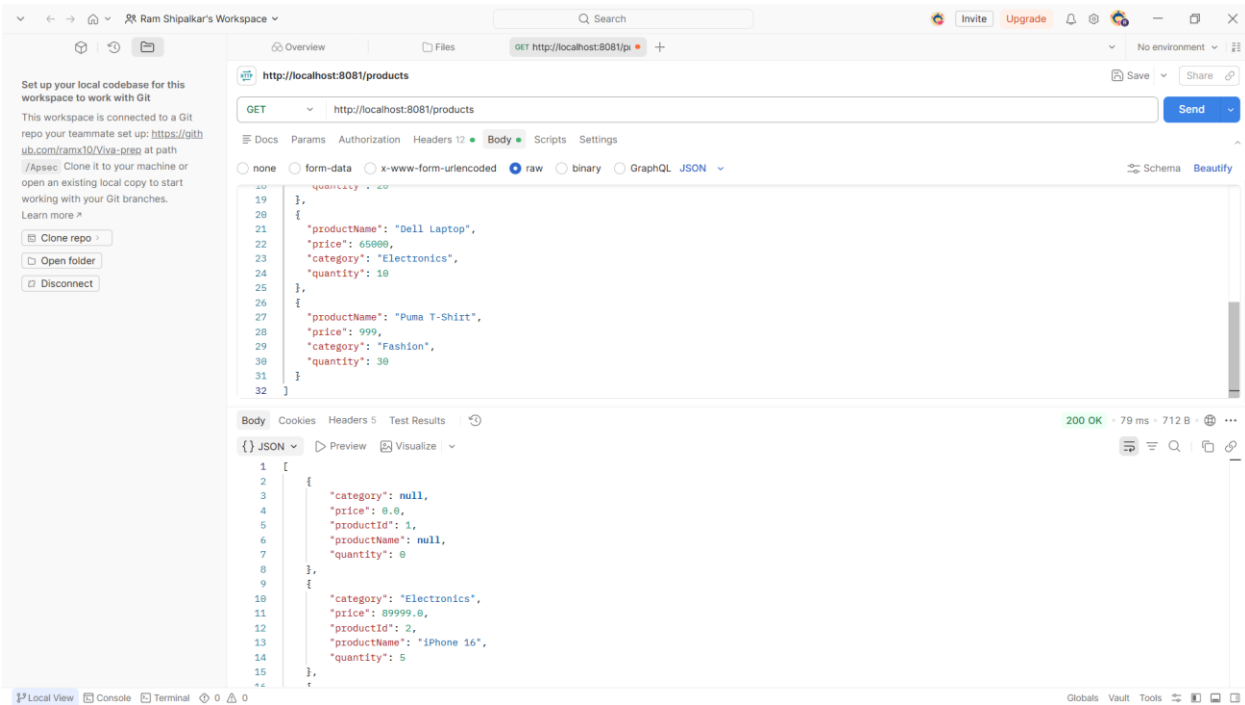


Figure 12 · GET /products — Retrieve all products · 200 OK

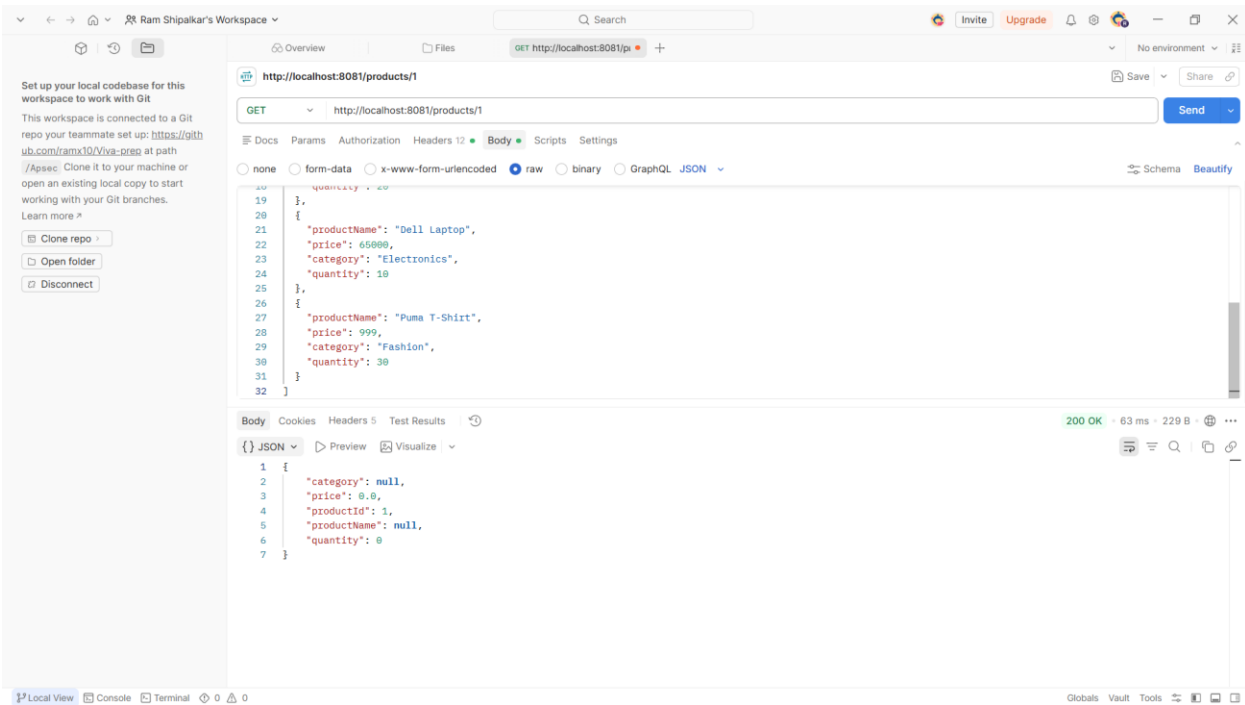


Figure 13 · GET /products/1 — Retrieve product by ID · 200 OK

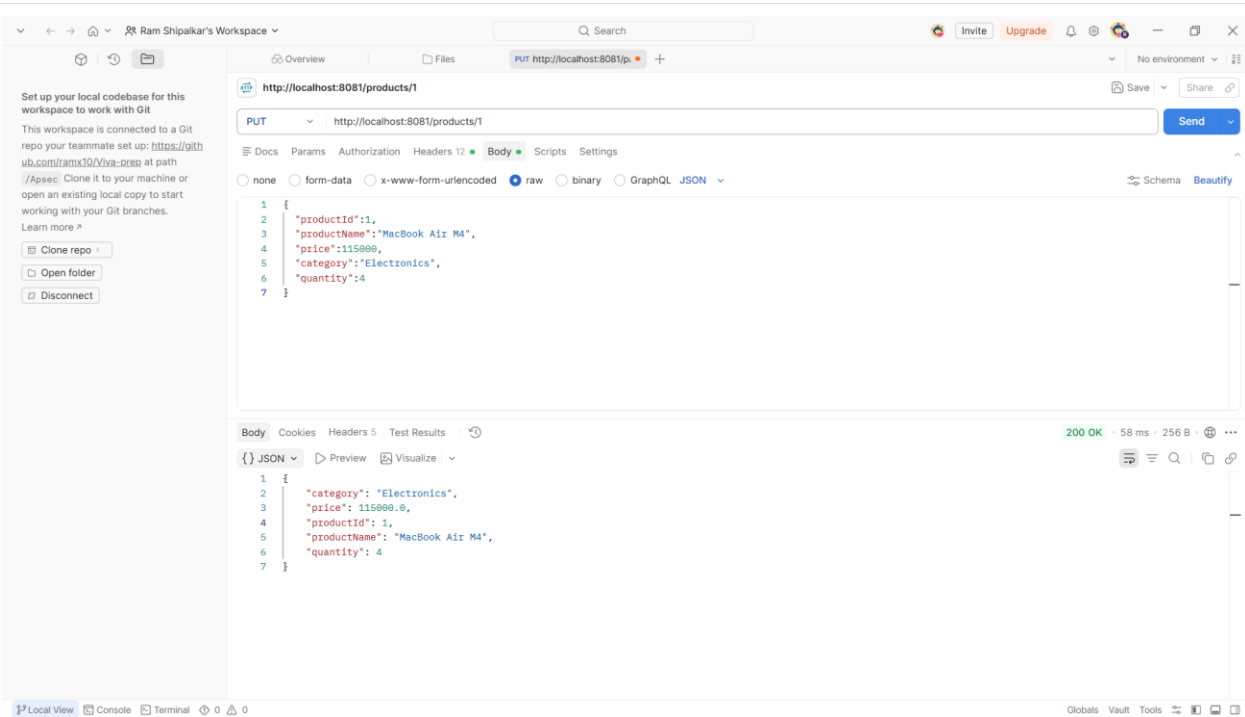


Figure 14 · `PUT /products/1` — Update product (MacBook Air M4, Rs. 115000) · 200 OK

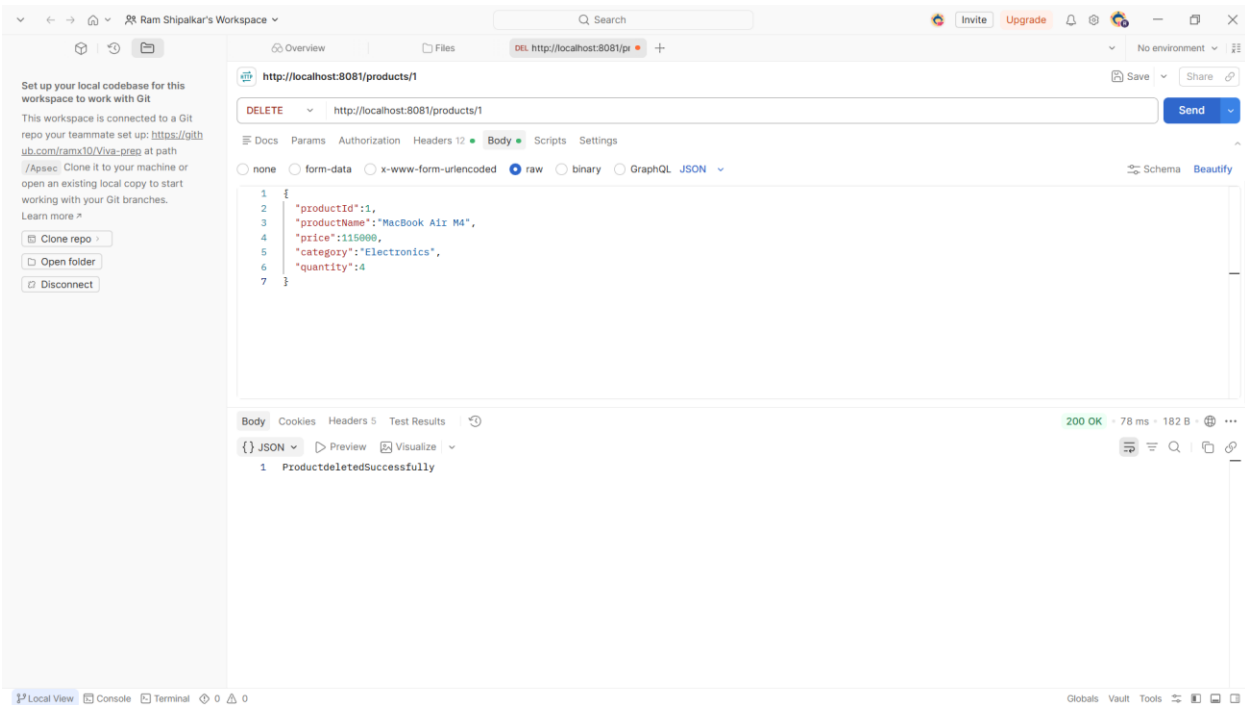


Figure 15 · `DELETE /products/1` — Delete product, response: `ProductdeletedSuccessfully` · 200 OK

Set up your local codebase for this workspace to work with Git. This workspace is connected to a Git repo your teammate set up: <https://github.com/ramx10/viva-rpg> at path `/Aspec`. Clone it to your machine or open an existing local copy to start working with your Git branches. Learn more >

Clone repo · Open folder · Disconnect

Overview | Files | GET http://localhost:8081/products

GET http://localhost:8081/products

Docs Params Authorization Headers 12 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
```

Body Cookies Headers 5 Test Results

```
{
  "category": "Electronics",
  "price": 89999.0,
  "productId": 2,
  "productName": "iPhone 16",
  "quantity": 5
},
{
  "category": "Electronics",
  "price": 79999.0,
  "productId": 3,
  "productName": "Samsung S25",
  "quantity": 8
}
```

200 OK · 79 ms · 636 B

Local View Console Terminal

Figure 16 · GET /products — All products after delete (product 1 removed) · 200 OK

REQUEST HISTORY

Today

- GET http://localhost:8081/products/...
- POST http://localhost:8081/products/...
- GET http://localhost:8081/products/...
- GET http://localhost:8081/products/...
- GET http://localhost:8081/products/...
- GET http://localhost:8081/products/...
- GET http://localhost:8081/products/...
- DEL http://localhost:8081/products/1
- PUT http://localhost:8081/products/1
- GET http://localhost:8081/products/1
- GET http://localhost:8081/products/1
- GET http://localhost:8081/products
- POST http://localhost:8081/products/...
- POST http://localhost:8081/products
- POST http://localhost:8081/pro...
- DEL http://localhost:8081/customer...
- DEL http://localhost:8081/customer...
- DELETE http://localhost:8081/c...
- PUT http://localhost:8081/customer...
- GET http://localhost:8081/customer...
- GET http://localhost:8081/customer...
- GET http://localhost:8081/customer...
- POST http://localhost:8081/customer...
- POST http://localhost:8081/customer...

Yesterday

- PUT http://localhost:8081/customer...
- GET http://localhost:8081/customer...

Overview | Files | GET http://localhost:8081/products/category/Electronics

GET http://localhost:8081/products/category/Electronics

Docs Params Authorization Headers 12 Body Scripts Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results

```
{
  "category": "Electronics",
  "price": 89999.0,
  "productId": 1,
  "productName": "iPhone 16",
  "quantity": 5
},
{
  "category": "Electronics",
  "price": 79999.0,
  "productId": 2,
  "productName": "Samsung S25",
  "quantity": 8
}
```

200 OK · 172 ms · 449 B

Local View Console Terminal

Figure 17 · GET /products/category/Electronics — Filter products by category · 200 OK

Orders API /orders

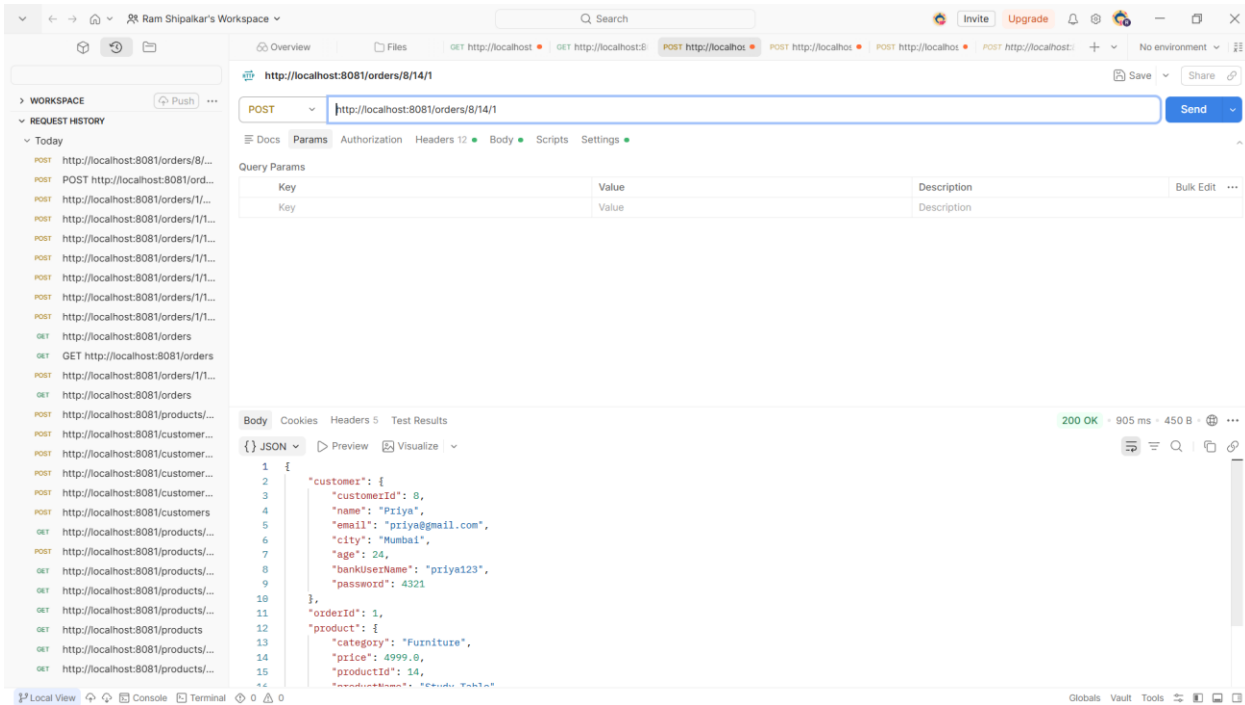


Figure 18 · POST /orders/8/14/1 — Place order (Customer: Priya, Product: Study Table, Qty: 1) · 200 OK

8. Repository — Custom Queries

Method	Repository	Type
<code>findByEmail(email)</code>	CustomerRepository	Derived query
<code>findByCity(city)</code>	CustomerRepository	Derived query
<code>findByAgeGreaterThanOrEqual(age)</code>	CustomerRepository	Derived query
<code>findByCategory(category)</code>	ProductRepository	Derived query
<code>countOrder(customerId)</code>	OrderRepository	@Query — JPQL
<code>totalAmount(customerId)</code>	OrderRepository	@Query — JPQL
<code>totalRevenue()</code>	OrderRepository	@Query — JPQL

9. Conclusion

The Swiggy Order Management System demonstrates a complete Spring Boot backend stack:

- Layered architecture with clean separation of Controller, Service, and Repository
- @ManyToOne JPA mappings maintaining relational integrity across three entities
- Mix of derived query methods and JPQL @Query for flexible data access
- Business logic (total price computation) contained in the service layer
- All endpoints verified with Postman; data confirmed via H2 console queries